

Documentação Técnica: Frontend do Gêmeo Digital IoT

Versão: 1

Tecnologias Core: HTML5, CSS3 (Variáveis Nativas), JavaScript (ES6+), Leaflet.js, Leaflet.heat

1. Visão Geral do Sistema

O frontend funciona como a camada de visualização em tempo real (Painel de Operações) de um ecossistema de Gêmeo Digital Urbano. Ele renderiza dados espaciais de sensores IoT distribuídos geograficamente, permitindo a análise de múltiplos atributos ambientais (Temperatura, Umidade, CO₂, Luminosidade e Ruído) sob duas abordagens de exibição: **Vetorial discreta** (Marcadores pontuais com pulso dinâmico) e **Contínua** (Mapa de calor gerado dinamicamente no cliente).

A interface adota uma estética *Cyberpunk/Sci-Fi Dark Mode*, otimizada para salas de controle, utilizando CSS Grid estruturado para manter as proporções estáticas e barras de rolagem independentes (`overflow: hidden` global).

2. Arquitetura e Componentes da Interface

A interface do aplicativo está dividida em 4 macroáreas estruturadas via `grid-template-areas`:

- Header (#tb):** Centraliza a identidade visual, telemetria rápida de sensores ativos contra alertas totais, indicador de status do servidor QGIS e o relógio de sincronismo do sistema.
- Sidebar (#sd):** Menu lateral retrátil contendo:
 - Controle de Camadas:** Alternadores booleanos para ligar/desligar o Mapa Base, Marcadores WFS e o Mapa de Calor.
 - Seletor de Atributos:** Introspecção automática das chaves de dados dos sensores que renderiza botões de controle com as médias globais calculadas dinamicamente.
 - Legenda Dinâmica:** Régua de gradiente CSS que muda suas cores internas e limites com base nas regras do atributo selecionado.
 - Lista de Sensores:** Listagem em tempo real com indicador visual de criticidade (`online`, `warning`, `alert`).
- Map Canvas (#mp):** Área ocupada pelo mapa Leaflet, contendo coordenadas de posicionamento capturadas em tempo real através do evento `mousemove`.
- Footer Telemetry (#st):** Barra inferior que monitora a saúde das conexões de infraestrutura de dados (FastAPI, QGIS Server, Modo WAL do GeoPackage e timestamps de escrita).

3. Detalhamento de Funções Core (JavaScript)

3.1. Motor de Calor Calibrado (`updateHeatmap`)

Diferente de mapas de calor comuns que usam contagem por densidade de pontos, esta função foi calibrada para traduzir a **intensidade física** da temperatura:

- **Lógica:** Ela itera sobre o array de sensores, captura o valor bruto de `temperatura`, subtrai o valor mínimo estipulado no dicionário de metadados (`AC.temperatura.min = 15`) e divide pela amplitude térmica total da escala ($\$40 - 15 = 25\$$).
- **Resultado:** Isso gera um valor de peso (`intensity`) normalizado estritamente entre `0.0` e `1.0`. O algoritmo passa esses dados limpos para a instância `L.heatLayer`, que plota um gradiente térmico realista (azul para frio, amarelo para moderado e vermelho denso para calor crítico).

3.2. Ciclo de Vida da Edição de Nomes (`initEditName`, `checkEditEnter`, `saveName`)

A funcionalidade de edição opera diretamente sobre a árvore do DOM sem quebrar o estado de escopo do mapa:

1. Ao clicar em "Editar Nome", `initEditName()` localiza o container do título do Popup aberto (`#pnm-${id}`) e injeta dinamicamente um elemento `<input>` com o valor atual, focando-o imediatamente.
2. O botão inferior altera seu estado lógico e assinatura de evento de `initEditName` para `saveName`.
3. A submissão aceita o clique físico ou a captura da tecla `Enter` via `checkEditEnter()`.
4. O método `saveName()` aplica o *trimming* na string, sobrescreve o valor na memória, reconverte o input em texto puro e chama `setPopupContent()` no marcador correspondente para atualizar a instância do Leaflet para os próximos cliques.

4. Guia de Integração com o Backend (FastAPI)

Para migrar esta interface estática baseada em Mock Data para uma aplicação dinâmica conectada à produção, siga as diretrizes abaixo.

4.1. Preparando o Backend FastAPI (Python)

Para que o frontend (JavaScript no navegador) consiga consumir os dados do seu backend local ou em nuvem, o FastAPI **obrigatoriamente** precisa ter o middleware de **CORS (Cross-Origin Resource Sharing)** configurado, do contrário o navegador bloqueará as requisições por segurança.

Implemente os seguintes endpoints no seu código Python:

```
Python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Dict, Any

app = FastAPI()
```

```

# Configuração de CORS para permitir requisições do Frontend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Altere para o endereço IP/Domínio do frontend em produção
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Modelo Pydantic para validação na edição do nome
class SensorUpdate(BaseModel):
    name: str

# Banco de dados simulado (Em produção, conecte à sua matriz .dyndb ou SQLite/PostGIS)
db_sensores = [
    {
        "id": "SNS-001", "name": "Sensor 1", "blk": "Centro Histórico",
        "lat": -12.9714, "lng": -38.5101,
        "data": {"temperatura": 31.4, "umidade": 74.2, "co2": 610, "luminosidade": 2800, "ruído":
72},
        "st": "online", "ts": "20:00"
    }
    # Demais sensores enumerados...
]

@app.get("/api/v1/sensors")
async def listar_sensores():
    """Retorna a lista de todos os sensores cadastrados com suas telemetrias."""
    return db_sensores

@app.put("/api/v1/sensors/{sensor_id}")
async def atualizar_nome_sensor(sensor_id: str, payload: SensorUpdate):
    """Atualiza o nome do sensor na base de dados e no GeoPackage."""
    for sensor in db_sensores:
        if sensor["id"] == sensor_id:
            sensor["name"] = payload.name

            # [Fluxo Crítico do Gêmeo Digital]
            # Insira aqui a sua chamada de escrita para salvar as alterações no arquivo .gpkg
            # de modo que o seu QGIS Server leia a alteração imediatamente.
            # ex: atualizar_geopackage(sensor_id, payload.name)

    return {"status": "sucesso", "sensor": sensor}

raise HTTPException(status_code=404, detail="Sensor não encontrado")

@app.get("/api/v1/status")

```

```

async def obter_status_sistema():
    """Retorna dados de integridade do backend e sincronia espacial."""
    import datetime
    return {
        "ultima_atualizacao": datetime.datetime.now().strftime("%d/%m/%Y %H:%M:%S"),
        "status": "sucesso",
        "gpkg": "WAL"
    }

```

4.2. Alterações Necessárias no Código do Frontend (JavaScript)

Abra o seu arquivo HTML e realize as seguintes substituições para conectar as funções visuais aos endpoints criados:

Passo 1: Limpar os dados estáticos e definir o endereço da API

No topo do bloco `<script>`, remova as atribuições estáticas das constantes `SENSORS` e `STATUS_JSON` e defina-as assim:

```

JavaScript
let SENSORS = []; // Inicia vazio, aguardando o backend
const API_URL = "http://localhost:8000/api/v1"; // Endereço padrão do FastAPI local

```

Passo 2: Substituir a inicialização por uma requisição assíncrona (`fetch`)

Crie uma função global chamada `loadData()` encarregada de ler os dados das rotas `/sensors` e `/status`:

```

JavaScript
async function loadData() {
    try {
        // Busca dados dos sensores
        const response = await fetch(`${API_URL}/sensors`);
        if (!response.ok) throw new Error("Erro ao buscar sensores.");
        SENSORS = await response.json();

        // Busca status operacional da infraestrutura
        const resStatus = await fetch(`${API_URL}/status`);
        const statusData = await resStatus.json();

        // Dispara a re-renderização completa dos componentes da UI com dados reais
        renderSensors();
        renderList();
        renderAttrBtns();
        renderLegend();
        topStats();

        // Atualiza elementos de telemetria do footer
        document.getElementById('sync-ts').textContent = statusData.ultima_atualizacao;
        document.getElementById('sync-s').textContent = statusData.status;
    }
}

```

```

    document.getElementById('sync-s').style.color = statusData.status === 'sucesso' ?
    'var(--gn)' : 'var(--rd)';

    } catch (error) {
        console.error("Falha na integração com o FastAPI:", error);
        document.getElementById('lmsg').textContent = "Erro na requisição de dados à API
REST.";
    }
}

```

Passo 3: Atualizar a persistência de edição substituindo a função **saveName**

Modifique a função **saveName** original para enviar uma requisição HTTP via método **PUT** contendo a string no corpo em formato JSON:

JavaScript

```

async function saveName(id) {
    const inp = document.getElementById(`inp-${id}`);
    if (!inp) return;
    const newName = inp.value.trim();

    if (newName) {
        try {
            // Dispara a requisição de escrita para o servidor FastAPI
            const response = await fetch(`${API_URL}/sensors/${id}`, {
                method: 'PUT',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ name: newName })
            });

            if (!response.ok) throw new Error("Falha ao salvar no banco de dados.");
            const resultado = await response.json();

            // Sincroniza a memória local com o dado retornado validado pelo servidor
            const s = SENSORS.find(x => x.id === id);
            if (s) s.name = resultado.sensor.name;

            // Atualiza os painéis visuais imediatamente
            renderList();

            const container = document.getElementById(`pnm-${id}`);
            if (container) container.textContent = s.name;

            const btn = document.getElementById(`be-${id}`);
            if (btn) {
                btn.textContent = "Editar Nome";
                btn.setAttribute("onclick", `initEditName('${id}')`);
            }
        }
    }
}

```

```

    if (MKS[id]) {
      MKS[id].mk.setPopupContent(puContent(s));
    }

    updateHeatmap(); // Força a atualização espacial das intensidades térmicas

    // Gera o timestamp local para feedback instantâneo de sincronia
    const d = new Date();
    document.getElementById('sync-ts').textContent =
`${String(d.getDate()).padStart(2,'0')}/${String(d.getMonth()+1).padStart(2,'0')}/2026
${String(d.getHours()).padStart(2,'0')}:${String(d.getMinutes()).padStart(2,'0')}:${String(d.getSeconds()).padStart(2,'0')}`;

  } catch (error) {
    alert("Não foi possível persistir a alteração do nome no servidor backend.");
    console.error(error);
  }
}
}
}

```

Passo 4: Atualizar o gatilho de inicialização do sistema

No final do seu arquivo, modifique o escopo do listener `DOMContentLoaded` para chamar o carregamento de dados da API em vez de funções internas isoladas:

JavaScript

/* — INIT

```

*/
document.addEventListener('DOMContentLoaded', () => {
  boot(); // Executa o Splash Screen Animado
  setTimeout(() => {
    initMap(); // Constrói a instância Leaflet e as definições do Heatmap
    loadData(); // Executa as chamadas assíncronas assinaladas acima
    tick(); // Inicia o contador do relógio global
    setInterval(tick, 1000);

    // Polling Opcional: Atualiza o estado dos sensores e mapa de calor
    // em background a cada 10 segundos
    setInterval(loadData, 10000);
  }, 400);
});

```

5. Fluxo de Operação do Gêmeo Digital Integrado

Com essa arquitetura implantada, seu sistema executa o seguinte pipeline:

1. O Navegador renderiza a tela e chama o FastAPI via `GET`.

2. O FastAPI lê o arquivo **.gpkg** ou banco de dados relacional e retorna os dados atualizados das coordenadas e atributos.
3. Ao editar e salvar o nome de um sensor, o frontend dispara um **PUT**.
4. O FastAPI processa o **PUT**, salva na tabela física de dados e dispara comandos internos para remontar as camadas do GeoPackage lido pelo **QGIS Server**.
5. Na próxima renderização ou requisição de mapa (*WMS/WFS*), todas as instâncias do ecossistema estarão unificadas com a nova alteração persistida.